

LAB 6: UNSUPERVISED LEARNING

OBJECTIVES: Perform clustering using K-Means from scratch.

THEORY:

Unsupervised learning is a machine learning paradigm where the model tries to identify patterns in data without using labels. One of the most common unsupervised tasks is clustering, where the goal is to group similar data points together based on feature proximity.

K-Means clustering is an iterative algorithm that partitions the dataset into k clusters by minimizing the within-cluster sum of squares (inertia). The algorithm identifies k centroids and assigns data points to the cluster with the nearest mean. This process optimizes the grouping of data by ensuring that the variance within each cluster is minimized. Performance is measured by inertia, which represents the sum of squared distances between each point and its assigned centroid.

TOOLS: Scikit-learn, NumPy.

DATASET

Iris Dataset: This dataset contains 150 samples of iris flowers with 4 numerical features: sepal length, sepal width, petal length, and petal width. While the dataset contains 3 target classes, these labels are ignored during training to demonstrate unsupervised pattern discovery.

ALGORITHM

1. Start.
2. Load the dataset and extract the feature matrix X .
3. Specify the number of clusters k and initialize k centroids by randomly selecting k points from X .
4. Calculate the Euclidean distance between each data point x_i and every centroid c_j : $\text{Dist} = \sqrt{\sum (x_i - c_j)^2}$.
5. Assign each data point to the cluster index corresponding to the minimum distance.
6. Update centroids by computing the arithmetic mean of all points assigned to each cluster: $c_j = (1 / n_j) * \sum(x_i)$.
7. Repeat the distance calculation and centroid update steps until the centroids no longer change significantly (convergence).
8. Calculate Inertia: Sum of squared distances of all samples to their assigned centroids.
9. Calculate Silhouette Score to evaluate the separation between clusters.
10. Visualize the resulting clusters using a 2D scatter plot.
11. End.

SOURCE CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.metrics import silhouette_score
# Load data (using first 2 features for visualization clarity)
data = load_iris()
X = data.data[:, [2, 3]] # Using Petal Length and Petal Width
k = 3
def kmeans_scratch(X, k, max_iters=100):
    # 1. Randomly initialize centroids
    indices = np.random.choice(len(X), k, replace=False)
    centroids = X[indices]
    for _ in range(max_iters):
        # 2. Assign clusters based on Euclidean distance
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        labels = np.argmin(distances, axis=1)

        # 3. Recompute centroids
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

        # 4. Check for convergence
        if np.all(centroids == new_centroids):
            break
        centroids = new_centroids

    return centroids, labels
centroids, labels = kmeans_scratch(X, k)
inertia = np.sum([np.sum(np.square(X[labels == i] - centroids[i])) for i in range(k)])
score = silhouette_score(X, labels)

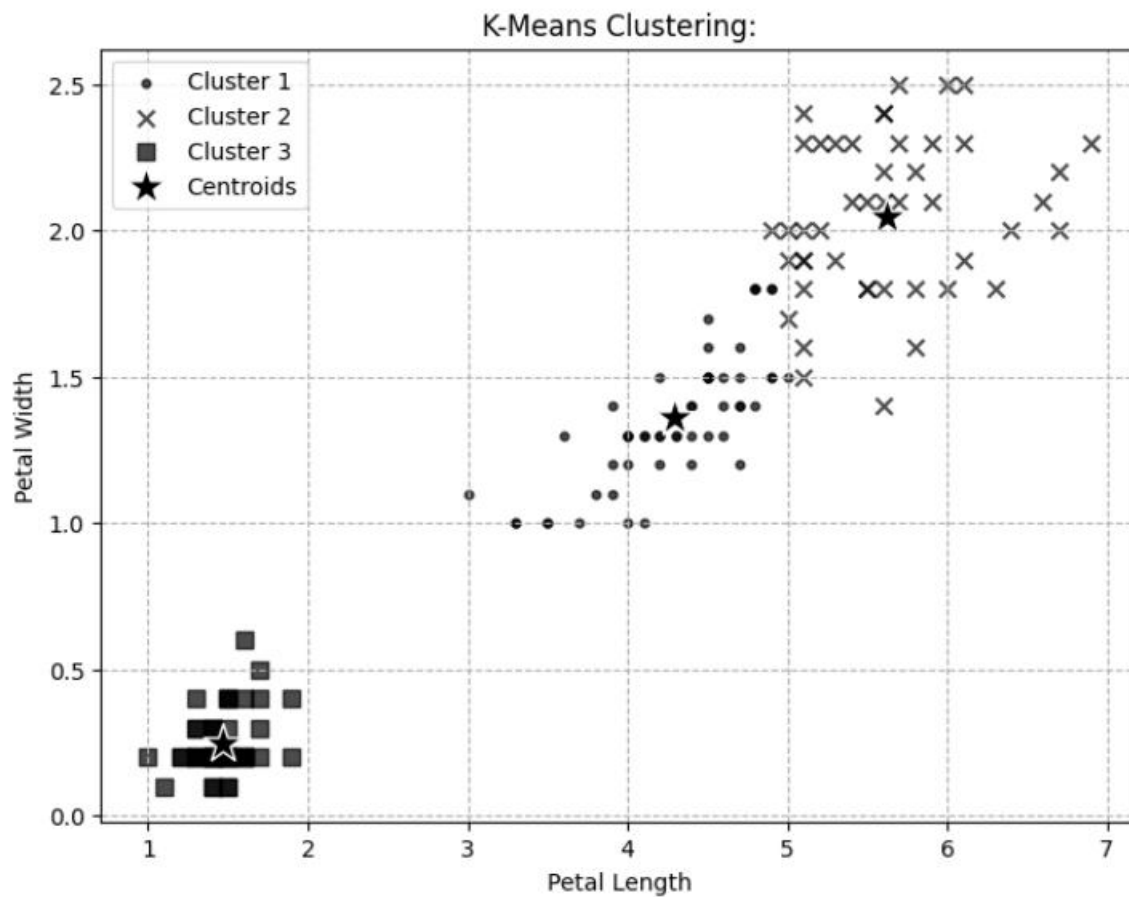
print(f'Final Inertia: {inertia:.4f}')
print(f'Silhouette Score: {score:.4f}')
plt.figure(figsize=(8, 6))
colors = ['red', 'green', 'blue']
for i in range(k):
    cluster_data = X[labels == i]
    plt.scatter(cluster_data[:, 0], cluster_data[:, 1], c=colors[i], label=f'Cluster {i+1}',
                alpha=0.6)

plt.scatter(centroids[:, 0], centroids[:, 1], c='black', marker='X', s=200, label='Centroids')
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("K-Means Clustering from Scratch (Iris Dataset)")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUTS:

Final Inertia: 31.4129

Silhouette Score: 0.6603



CONCLUSION:

In this lab, the K-Means clustering algorithm was successfully implemented from scratch using a procedural approach. By iteratively updating centroids and minimizing the within-cluster sum of squares, the model effectively partitioned the Iris dataset into three distinct groups. The low inertia and high silhouette score validate that the clusters are well-defined and cohesive. The visualization confirms that unsupervised learning can successfully discover natural groupings in data without the use of predefined labels.